



Uniwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych

PROGRAMOWANIE OBIEKTOWE

Testownik - aplikacja do zarządzania testami i ocenami

Michał Nocek

Nr albumu: 134951

Kierunek studiów: Informatyka

Praca wykonana pod kierunkiem
mgr inż. Ewa Żesławska

Rzeszów, 2026

Spis treści

1. Streszczenie w języku polskim i angielskim	6
1.1. Streszczenie w języku polskim.....	6
1.2. Streszczenie w języku angielskim	6
2. Opis założeń projektu	7
2.1. Wymagania funkcjonalne projektu	7
2.2. Wymagania нефункционалне projektu	8
3. Opis struktury projektu	10
3.1. Architektura i wykorzystane technologie	10
3.2. Zarządzanie danymi - baza danych	10
3.3. Hierarchia klas i kluczowe metody	12
3.4. Minimalne wymagania programu.....	14
3.5. Uruchomienie aplikacji	14
4. Harmonogram realizacji projektu	15
4.1. Napotkane trudności w trakcie realizacji projektu	15
4.2. Repozytorium i system kontroli wersji.....	16
5. Prezentacja warstwy użytkowej projektu.....	17
5.1. Ekran logowania i rejestracji	17
5.2. Pulpit nauczyciela.....	17
5.3. Lista testów i edytor testu.....	18
5.4. Ocenianie pytań otwartych	20
5.5. Zarządzanie grupami	20
5.6. Statystyki testu i eksport wyników	21
5.7. Lista testów ucznia	22
5.8. Rozwiązywanie testu	23
5.9. Wynik testu	23
6. Podsumowanie	25
Bibliografia	27
Spis rysunków	28
Spis tabel.....	29

1. Streszczenie w języku polskim i angielskim

1.1. Streszczenie w języku polskim

Dokument ten opisuje aplikację *Testownik* - system informatyczny służący do tworzenia, udostępniania i przeprowadzania testów oraz do zarządzania wynikami uczniów. Głównym celem projektu było dostarczenie nauczycielowi narzędzia, które automatyzuje cały cykl pracy z testem: od ułożenia pytań, przez przypisanie testu grupom uczniów, aż po automatyczne i ręczne ocenianie odpowiedzi oraz analizę statystyczną wyników.

Aplikacja została napisana w języku C# na platformie .NET z wykorzystaniem technologii *Windows Presentation Foundation* (WPF) do budowy interfejsu graficznego oraz *Entity Framework Core* jako warstwy dostępu do relacyjnej bazy danych PostgreSQL. System rozróżnia dwie role użytkowników - nauczyciela i ucznia - udostępniając każdej z nich odrębny zestaw funkcji. Nauczyciel tworzy testy złożone z pytań jednokrotnego wyboru, wielokrotnego wyboru oraz pytań otwartych, zarządza grupami uczniów i analizuje wyniki, natomiast uczeń rozwiązuje przypisane mu testy w ograniczonym czasie i przegląda swoje oceny. W aplikacji zaimplementowano między innymi limit podejść do testu, automatyczne ocenianie pytań zamkniętych, ręczne ocenianie i edycję oceny, sterowanie widocznością testów, wykrywanie prób oszustwa, eksport wyników do pliku CSV oraz haszowanie haseł.

1.2. Streszczenie w języku angielskim

This document describes the *Testownik* application - an information system for creating, sharing and conducting tests, as well as for managing students' results. The main goal of the project was to provide the teacher with a tool that automates the entire workflow of working with a test: from composing questions, through assigning the test to groups of students, up to automatic and manual grading of answers and the statistical analysis of results.

The application was written in C# on the .NET platform using *Windows Presentation Foundation* (WPF) for the graphical user interface and *Entity Framework Core* as the data access layer for a relational PostgreSQL database. The system distinguishes two user roles - the teacher and the student - providing each of them with a separate set of features. The teacher creates tests consisting of single-choice, multiple-choice and open questions, manages groups of students and analyses results, while the student solves the assigned tests within a limited time and reviews their grades. The application implements, among others, a limit on the number of attempts, automatic grading of closed questions, manual grading and grade editing, control over test visibility, cheating detection, exporting results to a CSV file and password hashing.

2. Opis założeń projektu

Celem projektu jest stworzenie aplikacji desktopowej wspierającej nauczyciela w tworzeniu i przeprowadzaniu testów oraz ucznia w ich rozwiązywaniu. Tradycyjne, papierowe sprawdzanie wiedzy jest czasochłonne - wymaga ręcznego przygotowania arkuszy, ich powielenia, a następnie żmudnego sprawdzania i podliczania punktów. Dodatkowo gromadzenie wyników i analiza, które zagadnienia sprawiają uczniom największą trudność, są w takim modelu praktycznie niemożliwe do przeprowadzenia na bieżąco.

Aplikacja *Testownik* rozwiązuje te problemy, przenosząc cały proces do jednego, spójnego narzędzia. Nauczyciel układa test w edytorze, przypisuje go wybranym grupom uczniów i otrzymuje automatycznie policzone wyniki wraz ze statystykami. Uczeń loguje się do systemu, rozwiązuje przypisane mu testy w ograniczonym czasie i natychmiast poznaje swój wynik. Realizacja projektu wymagała znajomości języka C# i programowania obiektowego, technologii WPF do budowy interfejsu graficznego, biblioteki Entity Framework Core do obsługi bazy danych, projektowania relacyjnych baz danych PostgreSQL oraz systemu kontroli wersji Git.

2.1. Wymagania funkcjonalne projektu

- **Zarządzanie kontami i rolami użytkowników**

- Użytkownik może samodzielnie zarejestrować nowe konto (imię, nazwisko, login, hasło), wybierając przy rejestracji rolę - ucznia lub nauczyciela - oraz zalogować się do systemu za pomocą loginu i hasła. Login jest unikalny w skali całego systemu.
- System rozróżnia dwie role - nauczyciela i ucznia - i po zalogowaniu udostępnia każdej z nich odrębny zestaw funkcji oraz inny układ nawigacji. Każdy nauczyciel zarządza wyłącznie własnymi testami.

- **Zarządzanie grupami uczniów**

- Nauczyciel może tworzyć nowe grupy (np. klasy) opatrzone nazwą i opisem.
- Nauczyciel może przypisywać uczniów do grup oraz usuwać ich z grup; jeden uczeń może należeć do wielu grup jednocześnie.
- System wyświetla skład każdej grupy oraz przypisane do niej testy.

- **Tworzenie i edycja testów**

- Nauczyciel tworzy testy złożone z pytań trzech typów: jednokrotnego wyboru, wielokrotnego wyboru oraz pytań otwartych.
- Dla każdego testu określa nazwę, przedmiot (wpisywany dowolnie lub wybierany z listy), limit czasu, próg zaliczenia (w procentach), dozwoloną liczbę podejść oraz grupy, którym test jest udostępniany.
- Dla każdego pytania określa treść, liczbę punktów oraz - w pytaniach zamkniętych - warianty odpowiedzi i poprawne odpowiedzi, a w pytaniach otwartych odpowiedź wzorcową.
- Test ma stan widoczności - aktywny lub nieaktywny. Tylko aktywny test jest widoczny dla uczniów; nowy test powstaje jako nieaktywny (szkic), a nauczyciel aktywuje go, gdy jest gotowy do przeprowadzenia.

- Test można edytować, duplikować oraz usuwać. Jeżeli test ma już rozwiązania uczniów, edycja pytań jest blokowana (edytowalne pozostają ustawienia i grupy), aby nie naruszyć powiązanych z pytaniami odpowiedzi.

- **Rozwiązywanie testów przez ucznia**

- Uczeń widzi listę testów przypisanych do jego grup, z podziałem na rozwiązane i nierozwiązane.
- Podczas rozwiązywania działa licznik czasu; po jego upływie test jest automatycznie kończony i oddawany.
- Uczeń może swobodnie przechodzić między pytaniami, a przy oddawaniu testu z pominiętymi pytaniami otrzymuje stosowne ostrzeżenie.
- Liczba podejść do testu jest ograniczona wartością ustawioną przez nauczyciela.

- **Ocenianie odpowiedzi**

- Pytania zamknięte są oceniane automatycznie - w pytaniach wielokrotnego wyboru z uwzględnieniem punktów częściowych.
- Pytania otwarte trafiają do kolejki oceniania, gdzie nauczyciel przyznaje punkty ręcznie, po czym wynik ucznia jest automatycznie przeliczany.
- Nauczyciel może ręcznie edytować ocenę dowolnego podejścia - przyznać lub nadpisać punkty każdego pytania, także automatycznie ocenionego pytania zamkniętego; wynik jest wówczas ponownie przeliczany.
- Przy wielu podejściach jako ostateczny brany jest najlepszy wynik ucznia.

- **Wykrywanie prób oszustwa**

- System wykrywa opuszczenie okna testu (np. przełączenie się na inną aplikację). Pierwsze opuszczenie skutkuje ostrzeżeniem, a kolejne - automatycznym zakończeniem testu z wynikiem 0% i oznaczeniem próby oszustwa.

- **Statystyki i eksport wyników**

- Nauczyciel ma dostęp do statystyk testu: zdawalności, średniego wyniku, najtrudniejszego pytania oraz poprawności w rozbiciu na poszczególne pytania.
- Wyniki testu można wyeksportować do pliku CSV.

2.2. Wymagania niefunkcjonalne projektu

System spełnia następujące kryteria jakościowe i ograniczenia:

- **Użyteczność** - interfejs użytkownika jest spójny i intuicyjny. Nawigacja odbywa się przez stały panel boczny, a poszczególne ekrany utrzymane są w jednolitej stylistyce (karty, znaczniki statusu, wykresy), co ułatwia poruszanie się po aplikacji.
- **Niezawodność i obsługa błędów** - operacje na bazie danych oraz operacje wejścia/wyjścia są zabezpieczone wspólnym mechanizmem obsługi wyjątków. W razie błędu użytkownik otrzymuje zrozumiały komunikat, a aplikacja nie ulega awarii. Dane wprowadzane w formularzach (np. ustawienia testu, kompletność pytań) są walidowane przed zapisem.

- **Wydajność** - kluczowe operacje, takie jak logowanie, wczytywanie testów czy zapis wyniku, wykonują się w czasie nieprzekraczającym kilku sekund, a uruchomienie aplikacji nie przekracza kilku sekund.
- **Bezpieczeństwo** - dostęp do funkcji systemu jest możliwy wyłącznie po zalogowaniu, a zakres uprawnień zależy od roli użytkownika. Hasła nie są przechowywane jawnie - zapisuje się ich skrót (SHA-256). Unikalność loginów jest wymuszana na poziomie bazy danych.
- **Środowisko** - aplikacja jest przeznaczona na system Windows z zainstalowanym środowiskiem uruchomieniowym .NET oraz dostępem do serwera bazy danych PostgreSQL.
- **Integralność danych** - spójność danych zapewniają klucze obce wraz z odpowiednio dobranymi regułami usuwania (kaskadowymi i ograniczającymi) oraz mechanizm migracji Entity Framework Core, wersjonujący schemat bazy danych.
- **Utrzymywalność** - kod źródłowy został napisany zgodnie z zasadami programowania obiektowego i podzielony na warstwy (model, dostęp do danych, logika/usługi, interfejs użytkownika), co ułatwia jego rozwój i modyfikację.

3. Opis struktury projektu

W tym rozdziale opisano, jak zbudowana jest aplikacja: jakie technologie zostały wykorzystane, jak zorganizowane są dane w bazie, jak wygląda hierarchia klas oraz jakie są minimalne wymagania potrzebne do uruchomienia programu.

3.1. Architektura i wykorzystane technologie

Aplikacja *Testownik* to program desktopowy napisany w języku C# [1] na platformie .NET. Jego budowa jest modularna i opiera się na podziale na warstwy, co zapewnia czytelność kodu oraz oddzielenie logiki od interfejsu użytkownika i od sposobu przechowywania danych. Wyróżnić można cztery główne warstwy:

- **Model (Models)** - zbiór klas encji odwzorowujących rzeczywiste obiekty dziedziny (użytkownik, grupa, test, pytanie, podejście, odpowiedź). Klasy te są mapowane na tabele bazy danych przez Entity Framework Core.
- **Dostęp do danych (Data)** - konfiguracja kontekstu bazy danych (*AppDbContext*), reguły mapowania relacji i dziedziczenia, mechanizm migracji oraz klasa zasilająca bazę danymi początkowymi (*DbSeeder*).
- **Logika i usługi (Services)** - warstwa pośrednia zawierająca repozytorium operacji bazodanowych (*DbRepository*), serwis oceniania (*GradingService*), eksport do CSV (*CsvExporter*), haszowanie haseł (*PasswordHash*) oraz wspólną obsługę wyjątków (*Safe*). W tej warstwie znajdują się także obiekty transferu danych (DTO), oddzielające encje bazodanowe od warstwy widoku.
- **Interfejs użytkownika (Views)** - ekrany aplikacji zbudowane w technologii WPF, odpowiedzialne za prezentację danych i obsługę interakcji z użytkownikiem. Widoki komunikują się z bazą wyłącznie poprzez warstwę usług.

Do budowy interfejsu graficznego wykorzystano technologię *Windows Presentation Foundation* (WPF) [3], która pozwala na tworzenie nowoczesnych, stylizowanych okien aplikacji desktopowych. Warstwę dostępu do danych zrealizowano przy użyciu biblioteki *Entity Framework Core* [2] - nowoczesnego mechanizmu mapowania obiektowo-relacyjnego (ORM), który umożliwia operowanie na bazie danych za pomocą obiektów języka C#, bez konieczności ręcznego pisania większości zapytań SQL. Jako silnik bazy danych zastosowano *PostgreSQL* [5], łączony z aplikacją za pośrednictwem dostawcy *Npgsql* [4]. Do kontroli wersji kodu wykorzystano system Git.

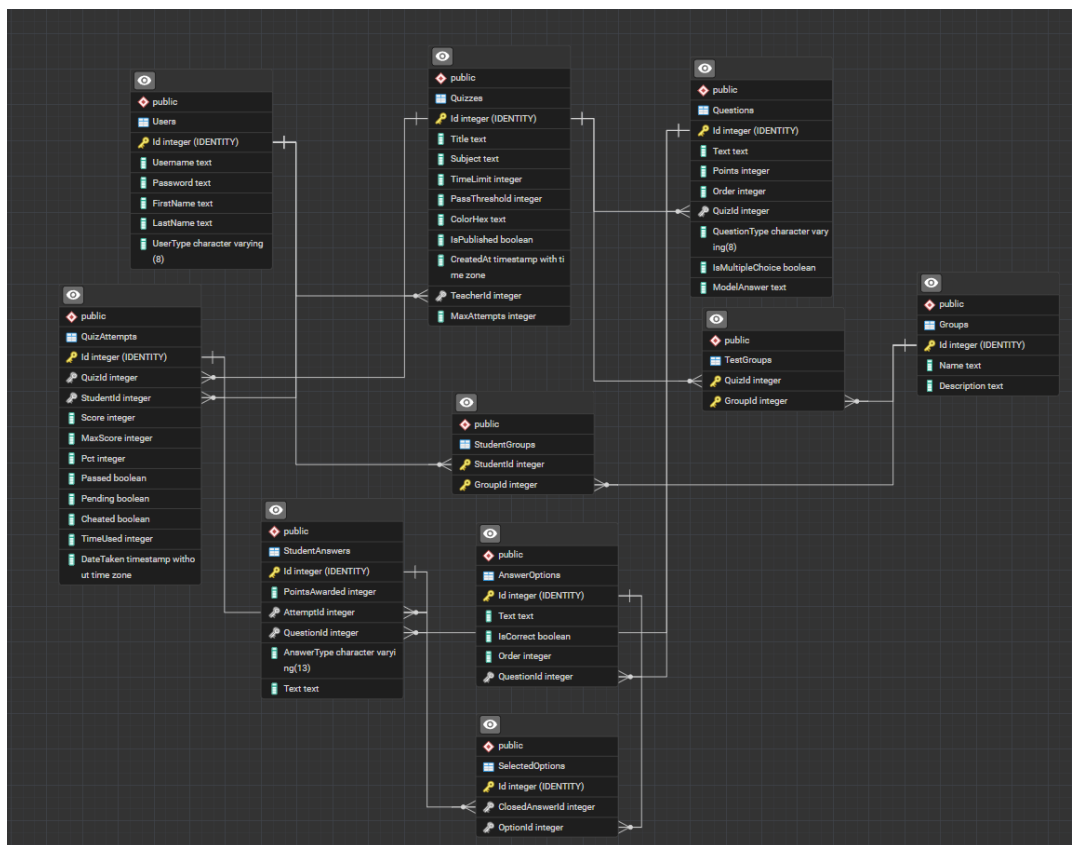
3.2. Zarządzanie danymi - baza danych

Wszystkie informacje o użytkownikach, grupach, testach, pytaniach, podejściach i odpowiedziach są przechowywane w relacyjnej bazie danych PostgreSQL. Schemat bazy jest tworzony i aktualizowany automatycznie za pomocą migracji Entity Framework Core, które wersjonują kolejne zmiany struktury. Najważniejsze tabele bazy danych przedstawiono w tabeli 3.1.

Tabela 3.1. Najważniejsze tabele bazy danych aplikacji.

Tabela	Przeznaczenie
Users	Konta użytkowników (wspólna tabela nauczycieli i uczniów, rozróżnianych dyskriminatorem); login, skrót hasła, imię i nazwisko.
Groups	Grupy uczniów (nazwa, opis).
StudentGroups	Tabela łącząca uczniów z grupami (relacja wiele-do-wielu).
Quizzes	Testy (nazwa, przedmiot, limit czasu, próg zaliczenia, liczba podejść).
Questions	Pytania (wspólna tabela pytań zamkniętych i otwartych); treść, punkty, kolejność.
AnswerOptions	Warianty odpowiedzi pytań zamkniętych; treść, flaga poprawności.
TestGroups	Tabela łącząca testy z grupami (relacja wiele-do-wielu).
QuizAttempts	Podejścia uczniów do testów; wynik, procent, status, czas, data.
StudentAnswers	Odpowiedzi ucznia (wspólna tabela odpowiedzi zamkniętych i otwartych).
SelectedOptions	Warianty zaznaczone przez ucznia w odpowiedzi zamkniętej.

Między tabelami zachodzą relacje jeden-do-wielu (np. test posiada wiele pytań, podejście posiada wiele odpowiedzi) oraz dwie relacje wiele-do-wielu, zrealizowane przez tabele łączące: uczeń może należeć do wielu grup, a grupa zrzeszać wielu uczniów (*StudentGroups*); analogicznie test może być przypisany do wielu grup, a grupa mieć wiele testów (*TestGroups*). Integralność danych zapewniają klucze obce wraz z regułami usuwania: kaskadowymi tam, gdzie dane podrzędne nie mają sensu bez nadrzędnych (usunięcie testu usuwa jego pytania i podejścia), oraz ograniczającymi tam, gdzie usunięcie mogłoby naruszyć już oddane odpowiedzi. Pełny schemat powiązań przedstawia diagram ERD na rysunku 3.1.

**Rysunek 3.1.** Diagram ERD bazy danych aplikacji.

3.3. Hierarchia klas i kluczowe metody

Projekt w szerokim zakresie wykorzystuje mechanizmy programowania obiektowego, w szczególności dziedziczenie. W modelu danych występują trzy hierarchie klas, mapowane na bazę danych strategią *Table-per-Hierarchy* (TPH), w której cała hierarchia jest zapisywana w jednej tabeli, a konkretny typ rozróżnia kolumna dyskryminatora.

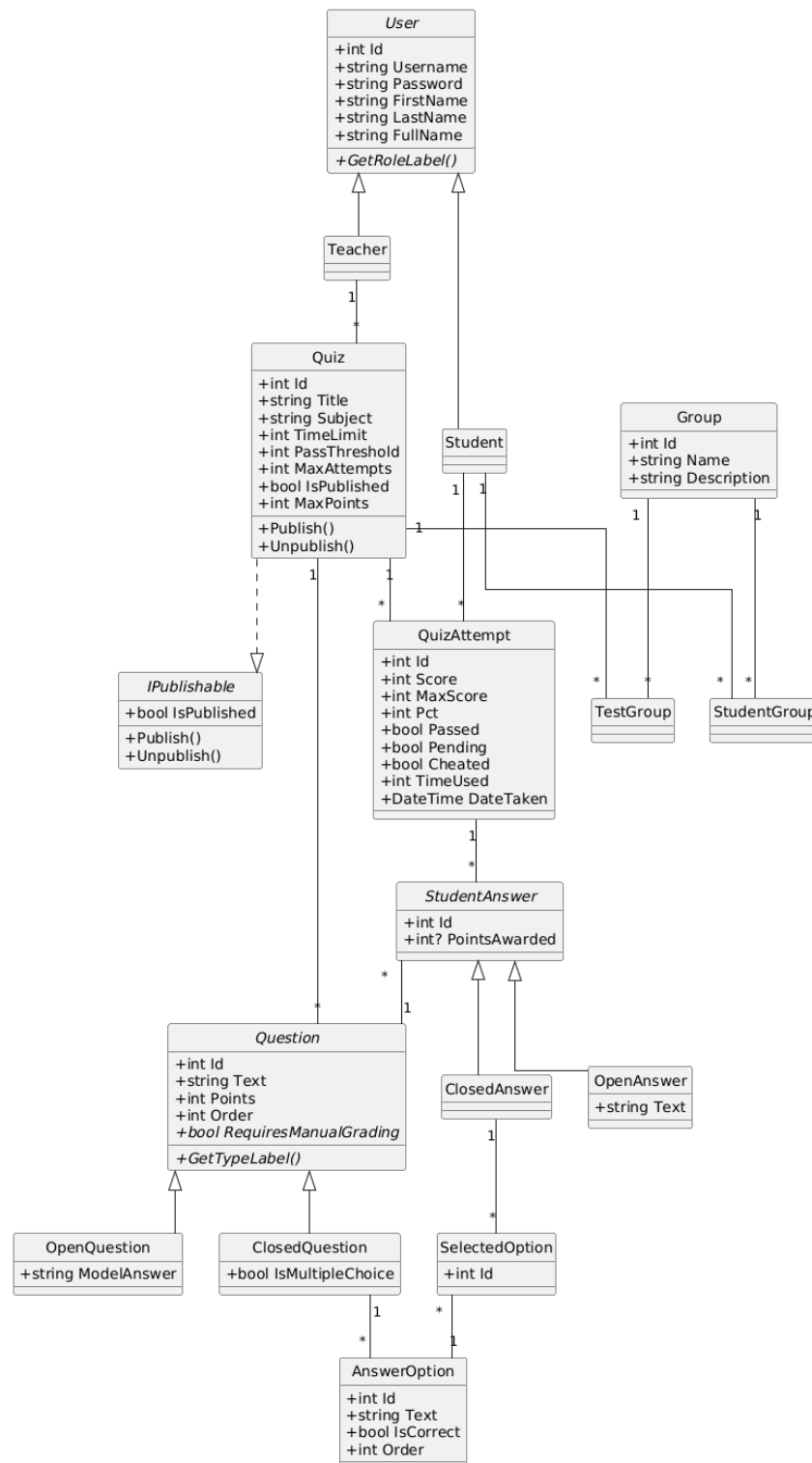
- **Hierarchia użytkowników** - abstrakcyjna klasa `User` przechowuje wspólne dane (identyfikator, login, skrót hasła, imię i nazwisko) oraz definiuje metodę abstrakcyjną `GetRoleLabel()`. Dziedziczą po niej klasy `Teacher` oraz `Student`. Uczeń posiada dodatkowo kolekcję grup, do których należy, oraz kolekcję swoich podejść do testów.
- **Hierarchia pytań** - abstrakcyjna klasa `Question` przechowuje treść, liczbę punktów i kolejność pytania. Dziedziczą po niej `ClosedQuestion` (pytanie zamknięte, posiadające warianty odpowiedzi oraz znacznik wielokrotnego wyboru) i `OpenQuestion` (pytanie otwarte z odpowiedzią wzorcową). Właściwość `RequiresManualGrading` rozróżnia, czy pytanie wymaga ręcznej oceny.
- **Hierarchia odpowiedzi** - abstrakcyjna klasa `StudentAnswer` jest bazą dla `ClosedAnswer` (przechowuje zaznaczone warianty) oraz `OpenAnswer` (przechowuje wpisany tekst). Pozwala to trzymać w jednej kolekcji odpowiedzi obu typów.

Oprócz dziedziczenia w projekcie wykorzystano interfejs `IPublishable`, implementowany przez klasę `Quiz`. Definiuje on właściwość `IsPublished` oraz metody `Publish()` i `Unpublish()`, służące do sterowania widocznością testu dla uczniów (stan aktywny i nieaktywny).

Logikę aplikacji realizują klasy warstwy usług:

- `DbRepository` - centralne repozytorium operacji na bazie danych. Hermetyzuje tworzenie kontekstu, pobieranie i zapisywanie testów, grup, podejść i wyników oraz mapowanie encji na obiekty transferu danych. Udostępnia m.in. metody zapisu nowego podejścia (`AddAttempt`), ręcznej aktualizacji oceny (`UpdateGrading`), pobrania najlepszego wyniku ucznia (`GetBestResult`), wyboru testów danego nauczyciela (`GetTestsForTeacher`), zmiany widoczności testu (`SetTestActive`) oraz bezpiecznego usuwania testu (`DeleteTest`).
- `GradingService` - statyczna klasa odpowiedzialna za ocenianie. Metoda `Grade` wyznacza liczbę punktów za pojedyncze pytanie (z punktami częściowymi w pytaniach wielokrotnego wyboru), a metoda `Recompute` przelicza cały wynik podejścia, uwzględniając pytania oczekujące na ręczną ocenę.
- `CsvExporter` - eksportuje wyniki testu do pliku CSV z użyciem strumieni (`StreamWriter`), dbając o poprawne kodowanie polskich znaków oraz cytowanie wartości.
- `PasswordHash` - wylicza skrót SHA-256 hasła, dzięki czemu hasła nie są przechowywane w bazie w postaci jawnej.
- `Safe` - wspólny mechanizm obsługi wyjątków; wykonuje operacje bazodanowe i wejścia/wyjścia w bloku zabezpieczającym, prezentując użytkownikowi czytelny komunikat zamiast powodować awarię aplikacji.

Wzajemne powiązania najważniejszych klas modelu przedstawia diagram UML na rysunku 3.2.



Rysunek 3.2. Diagram UML klas modelu aplikacji.

3.4. Minimalne wymagania programu

- **System operacyjny:** Windows 10 lub Windows 11.
- **Środowisko uruchomieniowe:** platforma .NET (wersja zgodna z projektem) wraz ze wsparciem WPF.
- **Serwer bazy danych:** PostgreSQL (zalecana wersja 13 lub nowsza), działający lokalnie lub zdalnie, z dostępem do bazy aplikacji.
- **Dodatkowe narzędzia:** środowisko programistyczne wspierające .NET (np. Visual Studio), system kontroli wersji Git oraz narzędzie do zarządzania bazą danych (np. pgAdmin).

3.5. Uruchomienie aplikacji

Aplikacja łączy się z bazą danych PostgreSQL przy użyciu parametrów zdefiniowanych w klasie `AppDbContext`: `serwer localhost`, `port 5432`, `baza testownik`, `użytkownik postgres`. Przed pierwszym uruchomieniem należy zapewnić działający serwer PostgreSQL z dostępem do tych parametrów.

Schemat bazy danych zakładany jest automatycznie. Przy starcie aplikacja wywołuje metodę `Database.Migrate()`, która tworzy bazę i nakłada wszystkie migracje Entity Framework Core, a następnie klasa `DbSeeder` wypełnia bazę danymi początkowymi (przykładowy nauczyciel, uczniowie, grupy oraz testy). Ręczne tworzenie tabel nie jest więc wymagane.

Kolejne kroki uruchomienia:

1. uruchomić serwer PostgreSQL i upewnić się, że jest dostępny na `localhost:5432`;
2. otworzyć projekt w środowisku Visual Studio i uruchomić aplikację (lub wykonać polecenie `dotnet run` w katalogu projektu);
3. przy pierwszym uruchomieniu baza zostanie utworzona i zasilona danymi demonstracyjnymi;
4. zalogować się przy użyciu jednego z kont demonstracyjnych.

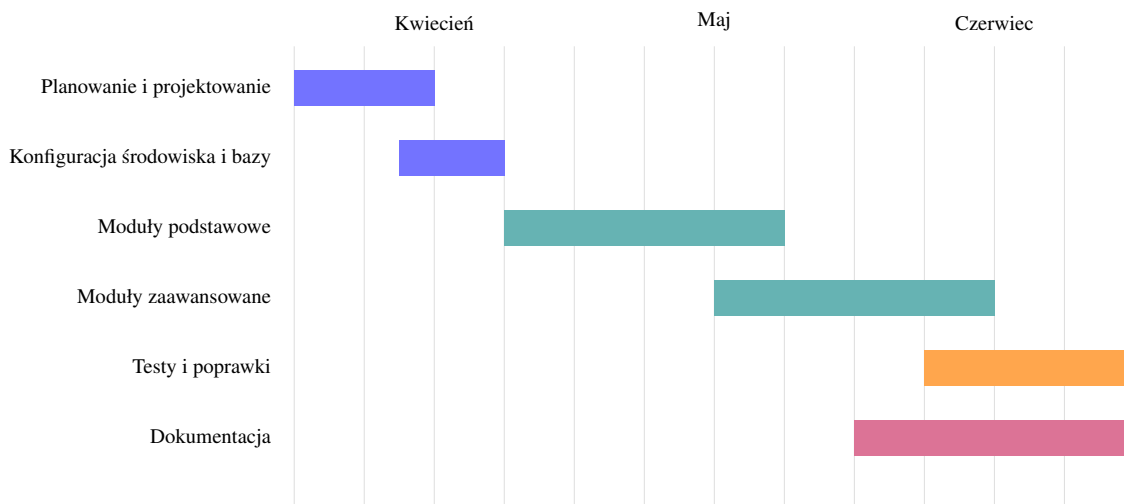
Konta demonstracyjne (hasło dla wszystkich: `demo1234`):

- nauczyciel - login `m.dabrowski`;
- uczniowie - loginy `anna.k`, `piotr.n`, `maria.w` oraz kolejni.

4. Harmonogram realizacji projektu

W tym rozdziale przedstawiono harmonogram realizacji projektu, obrazujący poszczególne etapy pracy oraz ich ramy czasowe. Do jego wizualizacji wykorzystano diagram Gantta (rysunek 4.1), który pozwala w czytelny sposób przedstawić planowanie i nakładanie się poszczególnych faz prac.

- **Planowanie i projektowanie** -faza początkowa obejmująca analizę wymagań oraz zaprojektowanie ogólnej struktury aplikacji i schematu bazy danych.
- **Konfiguracja środowiska i bazy danych** -przygotowanie narzędzi, utworzenie projektu oraz konfiguracja połączenia z bazą danych i pierwszych migracji.
- **Implementacja modułów podstawowych** -rozwój podstawowych funkcji: logowania i rejestracji, zarządzania grupami oraz tworzenia testów.
- **Implementacja modułów zaawansowanych** -ocenianie odpowiedzi, obsługa wielu podejść, wykrywanie oszustw oraz statystyki i eksport wyników.
- **Testy i poprawki** -weryfikacja poprawności działania aplikacji oraz eliminacja napotkanych błędów.
- **Tworzenie dokumentacji** -przygotowanie dokumentacji, prowadzone częściowo równoległe z pracami implementacyjnymi.



Rysunek 4.1. Diagram Gantta przedstawiający harmonogram realizacji projektu.

4.1. Napotkane trudności w trakcie realizacji projektu

Podczas realizacji projektu napotkano kilka trudności, które wymagały dodatkowego czasu i uwagi:

- **Mapowanie dziedziczenia na bazę danych** -odwzorowanie trzech hierarchii klas (użytkownicy, pytania, odpowiedzi) w bazie wymagało prawidłowej konfiguracji strategii Table-per-Hierarchy oraz dyskryminatorów w Entity Framework Core.

- **Reguły usuwania powiązanych danych** -konieczne było staranne dobranie reguł kaskadowego i ograniczającego usuwania, tak aby z jednej strony usunięcie testu czyściło powiązane dane, a z drugiej - edycja testu z istniejącymi rozwiązaniami nie naruszała oddanych już odpowiedzi uczniów.
- **Obsługa wielu podejść i wyboru najlepszego wyniku** -wymagała rozróżnienia między zapisem nowego podejścia a aktualizacją oceny istniejącego oraz spójnego liczenia wyników po stronie ucznia i nauczyciela.
- **Wykrywanie prób oszustwa** -konieczne było prawidłowe rozpoznawanie utraty fokusu okna oraz odróżnienie własnych okien dialogowych aplikacji od faktycznego przełączenia się użytkownika na inny program.

4.2. Repozytorium i system kontroli wersji

Kod źródłowy projektu jest przechowywany i zarządzany za pomocą systemu kontroli wersji Git. Dostęp do niego możliwy jest poprzez publiczne repozytorium w serwisie GitHub. Korzystanie z systemu kontroli wersji pozwoliło na śledzenie wszystkich zmian w kodzie oraz zarządzanie kolejnymi wersjami projektu.

Adres repozytorium projektu:

<https://github.com/MichalNocek/ProjektPO2>

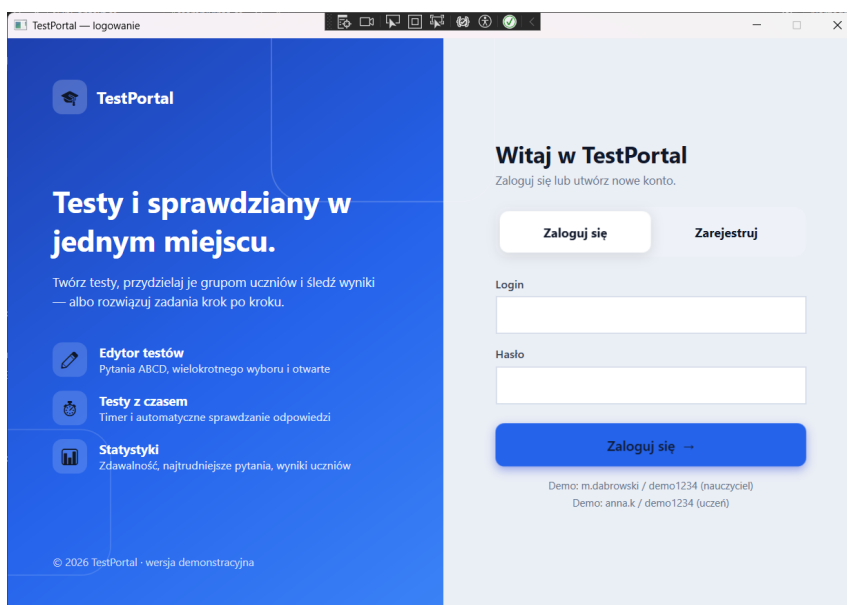
5. Prezentacja warstwy użytkowej projektu

W tym rozdziale przedstawiono wizualną stronę aplikacji za pomocą zrzutów ekranu, ilustrujących kluczowe widoki oraz interakcje użytkownika. Celem jest zaprezentowanie funkcjonalności systemu z perspektywy użytkownika końcowego - zarówno nauczyciela, jak i ucznia.

5.1. Ekran logowania i rejestracji

Pierwszym ekranem, z którym styka się użytkownik, jest ekran logowania, umożliwiający dostęp do systemu zarejestrowanym użytkownikom lub utworzenie nowego konta (ucznia lub nauczyciela). Interfejs przedstawiono na rysunku 5.1. Składa się on z:

- pola *Login*, służącego do wprowadzenia nazwy użytkownika,
- pola *Hasło*, zapewniającego maskowanie wpisywanego hasła,
- przycisku *Zaloguj się*, który weryfikuje podane dane z bazą danych i w przypadku ich poprawności przenosi użytkownika do odpowiedniego panelu (nauczyciela lub ucznia),
- zakładki *Rejestracja*, umożliwiającej utworzenie nowego konta po podaniu imienia, nazwiska, loginu i hasła oraz wybraniu roli (ucznia lub nauczyciela),
- komunikatów zwrotnych, informujących o błędnych danych logowania lub o problemach z walidacją formularza rejestracji.

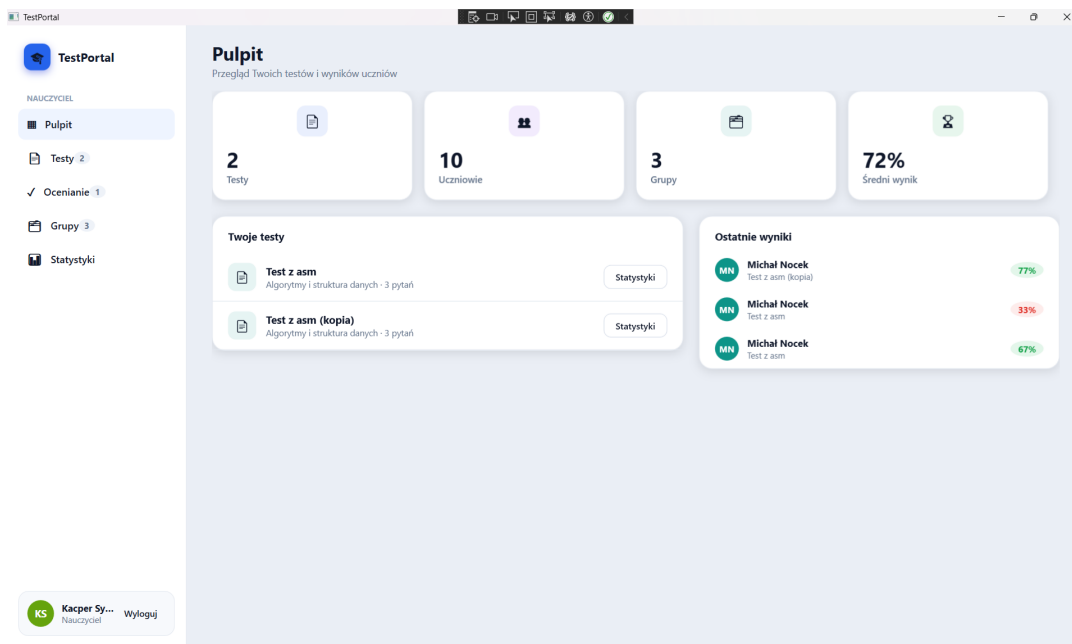


Rysunek 5.1. Ekran logowania i rejestracji.

5.2. Pulpit nauczyciela

Po zalogowaniu na konto nauczyciela wyświetlany jest pulpit (rysunek 5.2), stanowiący przegląd najważniejszych informacji. Zawiera on:

- kafelki podsumowujące liczbę testów, uczniów i grup oraz średni wynik,
- listę testów nauczyciela z szybkim przejściem do ich statystyk,
- panel ostatnich wyników uczniów,
- boczny panel nawigacji umożliwiający przejście do pozostałych modułów: testów, oceniania, grup i statystyk.

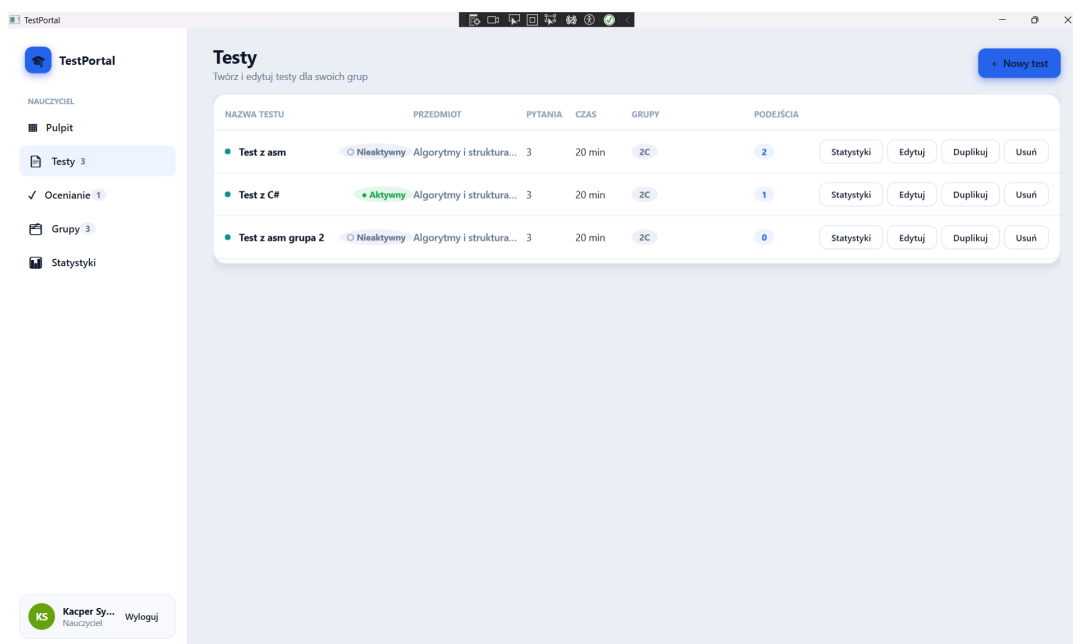


Rysunek 5.2. Pulpit nauczyciela.

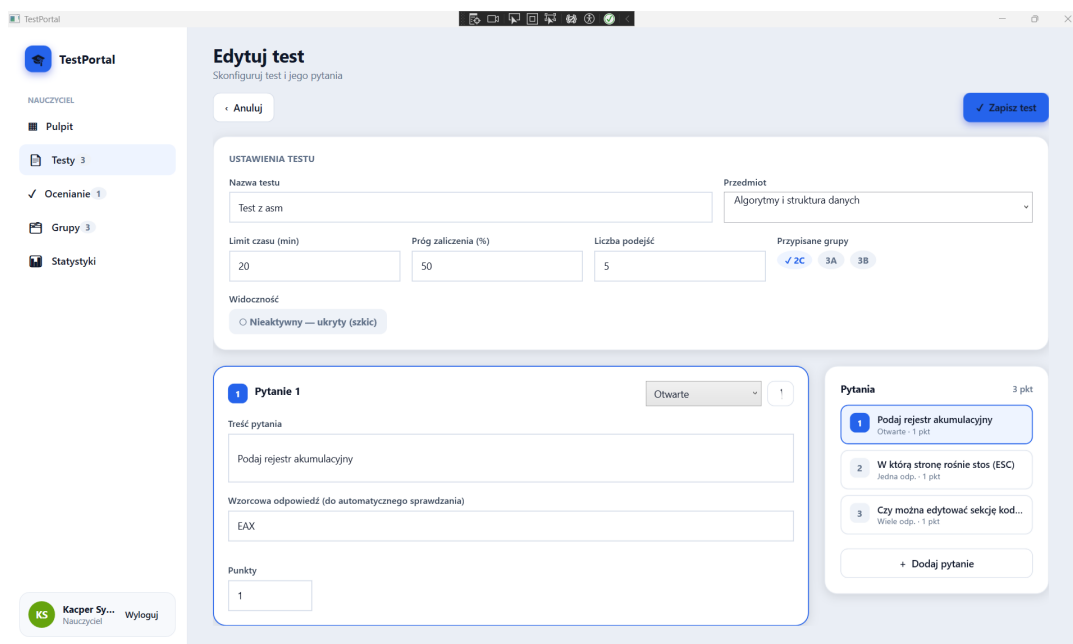
5.3. Lista testów i edytor testu

Widok *Testy* prezentuje wszystkie testy nauczyciela w formie tabeli z informacjami o nazwie i statusie widoczności (aktywny/nieaktywny), przedmiocie, liczbie pytań, czasie, przypisanych grupach i liczbie podejść. Status można przełączać jednym kliknięciem, a przy każdym teście dostępne są akcje: statystyki, edycja, duplikowanie oraz usunięcie. Z poziomu tego widoku nauczyciel tworzy nowy test lub otwiera istniejący w edytorze (rysunek 5.4).

Edytor testu pozwala określić ustawienia testu (nazwę, przedmiot wpisywany dowolnie lub wybierany z listy, limit czasu, próg zaliczenia, liczbę podejść, widoczność dla uczniów oraz przypisane grupy) i zbudować listę pytań. Dla każdego pytania można wybrać typ (jednokrotny wybór, wielokrotny wybór lub pytanie otwarte), wpisać treść, ustalić liczbę punktów oraz - w pytaniach zamkniętych - warianty odpowiedzi i poprawne odpowiedzi. Edytor waliduje kompletność testu przed zapisem (m.in. obecność poprawnej odpowiedzi i przypisanej grupy). Jeżeli test ma już rozwiązania, edycja pytań jest blokowana, o czym informuje stosowny komunikat.



Rysunek 5.3. Lista testów nauczyciela ze statusem widoczności i akcjami.



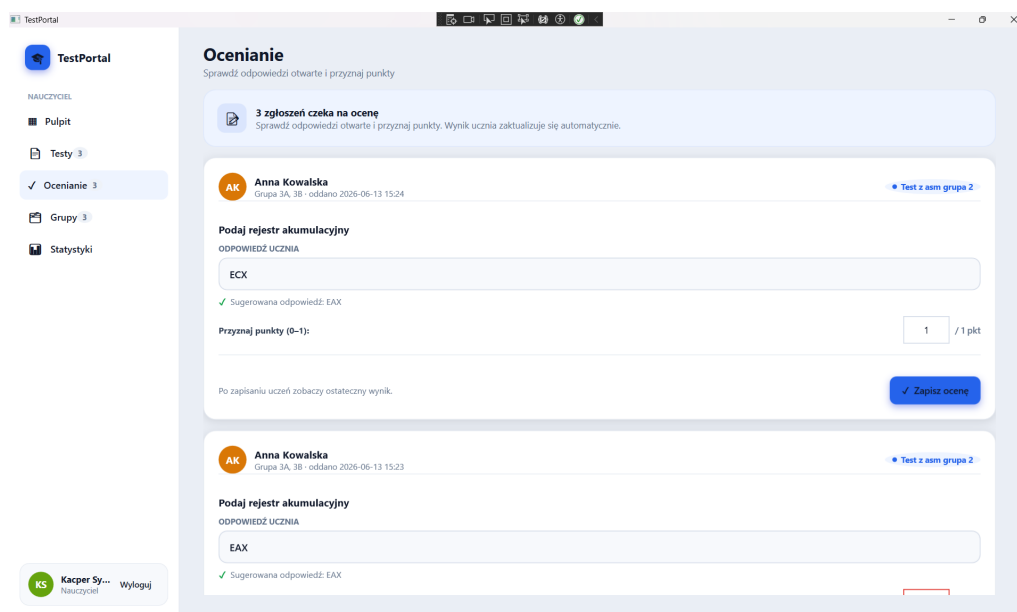
Rysunek 5.4. Edytor testu wraz z ustawieniami i listą pytań.

5.4. Ocenianie pytań otwartych

Moduł *Ocenianie* (rysunek 5.5) gromadzi podejścia uczniów zawierające pytania otwarte oczekujące na ręczną ocenę. Dla każdego zgłoszenia nauczyciel widzi:

- dane ucznia oraz nazwę testu i datę oddania,
- treść pytania otwartego oraz odpowiedź wpisaną przez ucznia,
- podpowiedź w postaci odpowiedzi wzorcowej,
- pole do wpisania przyznanych punktów w dozwolonym zakresie (z walidacją zakresu).

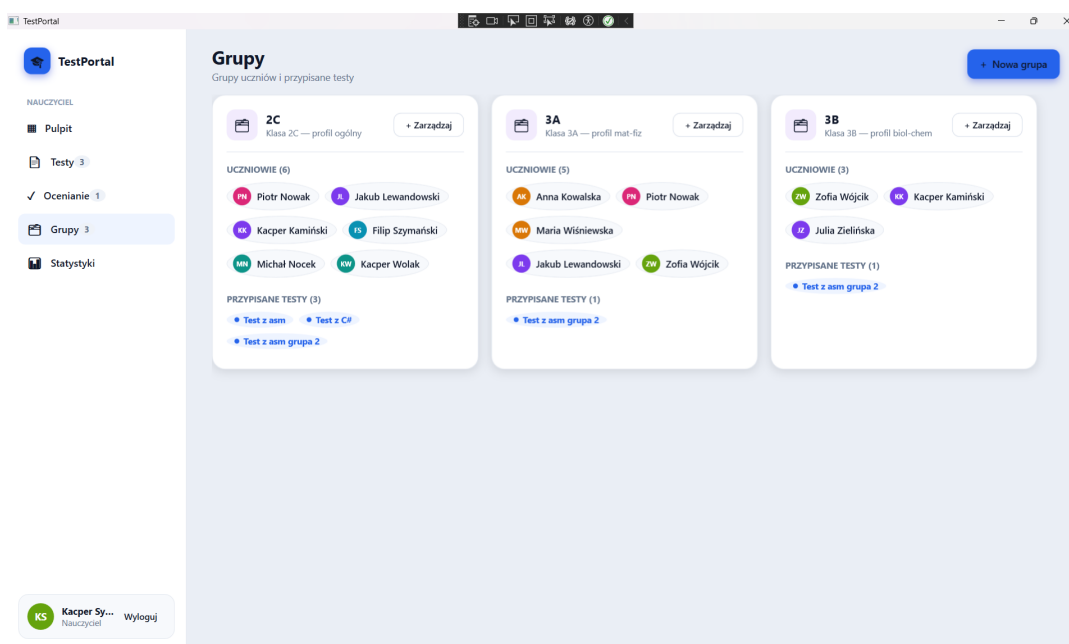
Po przyznaniu punktów wszystkim pytaniom otwartym i zapisaniu oceny wynik ucznia jest automatycznie przeliczany, a zgłoszenie znika z kolejki.



Rysunek 5.5. Ocenianie pytań otwartych.

5.5. Zarządzanie grupami

Widok *Grupy* (rysunek 5.6) prezentuje grupy uczniów w formie kart. Każda karta zawiera nazwę i opis grupy, listę przypisanych uczniów oraz listę testów udostępnionych grupie. Przycisk zarządzania otwiera okno, w którym nauczyciel zaznacza uczniów należących do grupy. Dodatkowo, z poziomu tego widoku można utworzyć nową grupę.



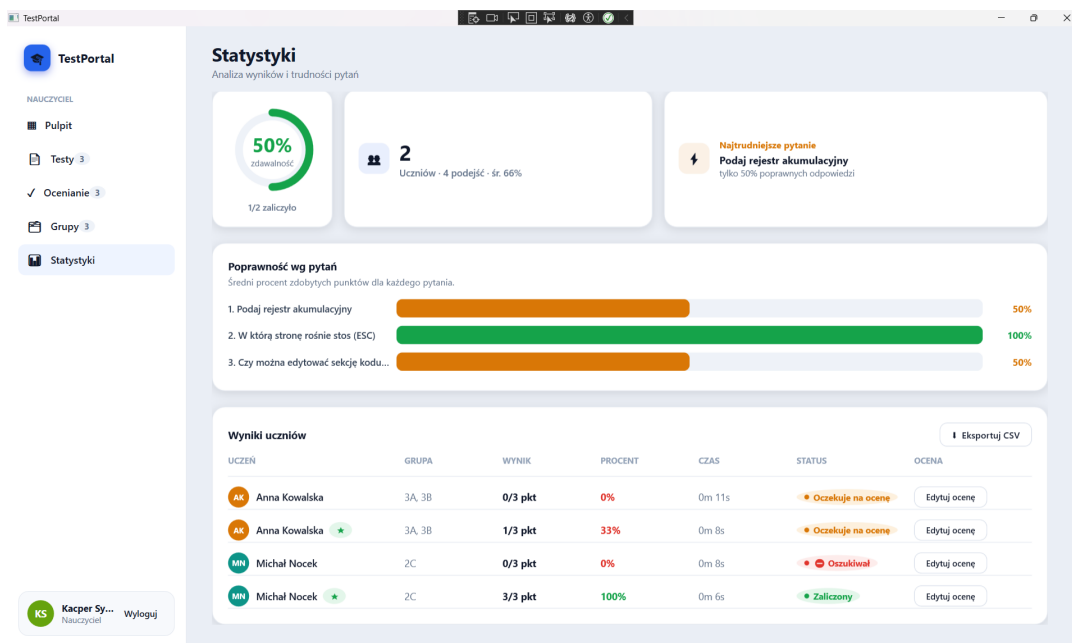
Rysunek 5.6. Widok zarządzania grupami uczniów.

5.6. Statystyki testu i eksport wyników

Moduł *Statystyki* (rysunek 5.7) prezentuje analizę wyników wybranego testu. Zawiera:

- wskaźnik zdawalności oraz liczbę uczniów, łączną liczbę podejść i średni wynik,
- informację o najtrudniejszym pytaniu,
- wykres poprawności w rozbiciu na poszczególne pytania,
- tabelę z wszystkimi podejściami uczniów (najlepsze podejście oznaczone gwiazdką) wraz ze statusem (zaliczony, niezaliczony, oczekujący na ocenę, próba oszustwa) oraz możliwością ręcznej edycji oceny każdego podejścia,
- przycisk eksportu wyników do pliku CSV.

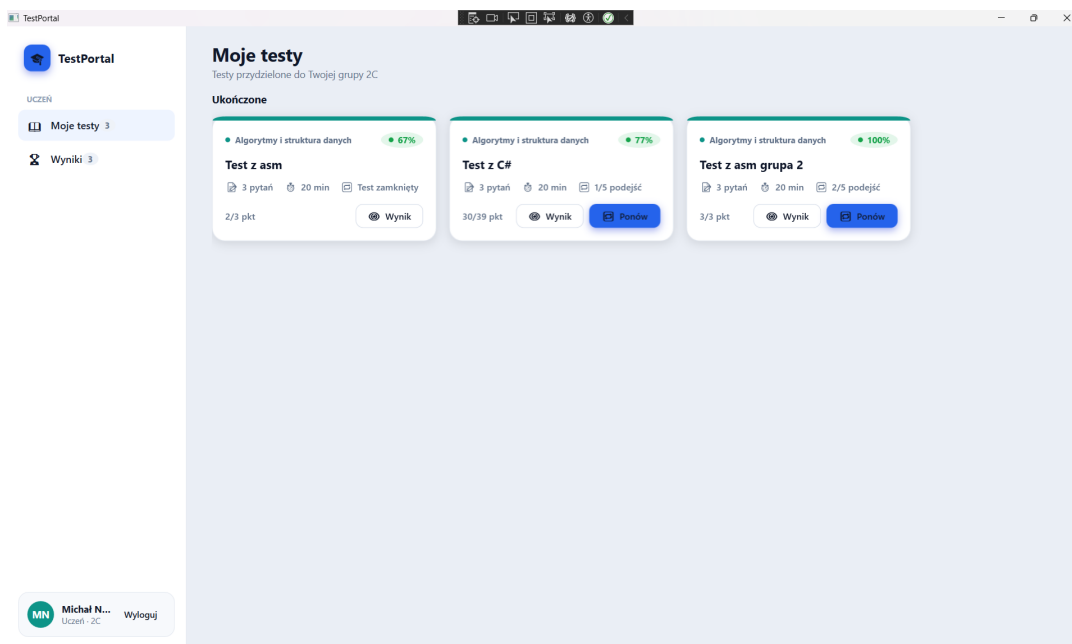
Wskaźniki zbiorcze (zdawalność, średni wynik) liczone są z najlepszego podejścia każdego ucznia.



Rysunek 5.7. Statystyki wyników testu.

5.7. Lista testów ucznia

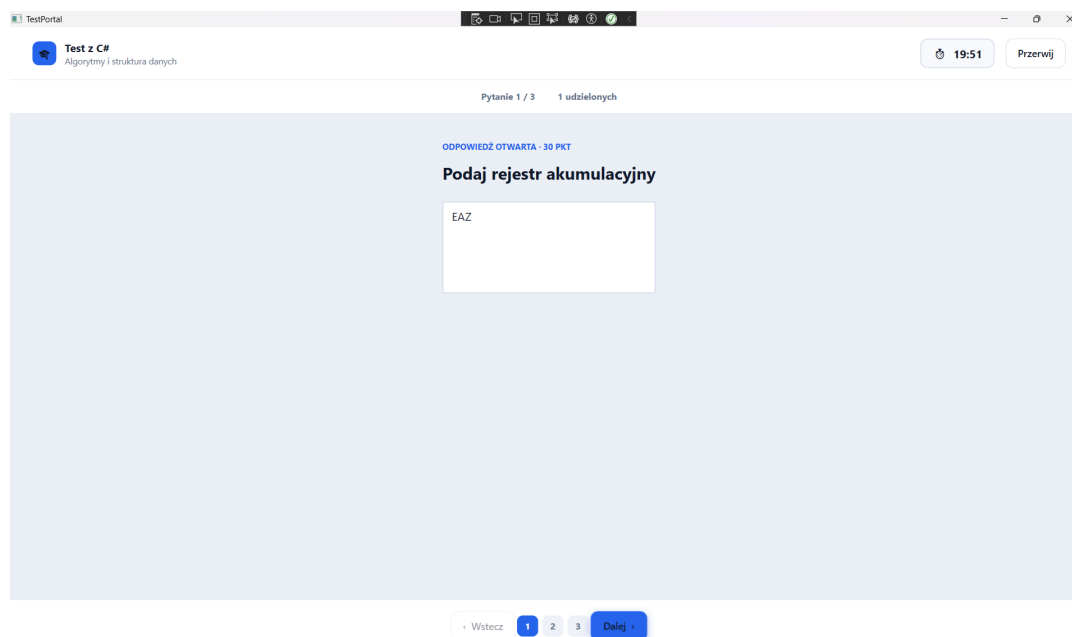
Po zalogowaniu na konto ucznia wyświetlana jest lista testów przypisanych do jego grup (rysunek 5.8), z podziałem na testy do zrobienia oraz ukończone. Każdy test prezentowany jest w formie karty z informacją o przedmiocie, liczbie pytań, czasie, liczbie wykorzystanych podejść oraz - dla testów rozwiązanych - z osiągniętym wynikiem i możliwością jego podglądu lub ponownego podejścia (jeśli limit nie został wyczerpany, a test jest nadal aktywny). Uczeń widzi wyłącznie testy aktywne oraz te nieaktywne, które już rozwiązał (zachowując dostęp do swojego wyniku); po dezaktywacji testu niewykorzystane podejścia nie są już dostępne.



Rysunek 5.8. Lista testów przypisanych uczniowi.

5.8. Rozwiązywanie testu

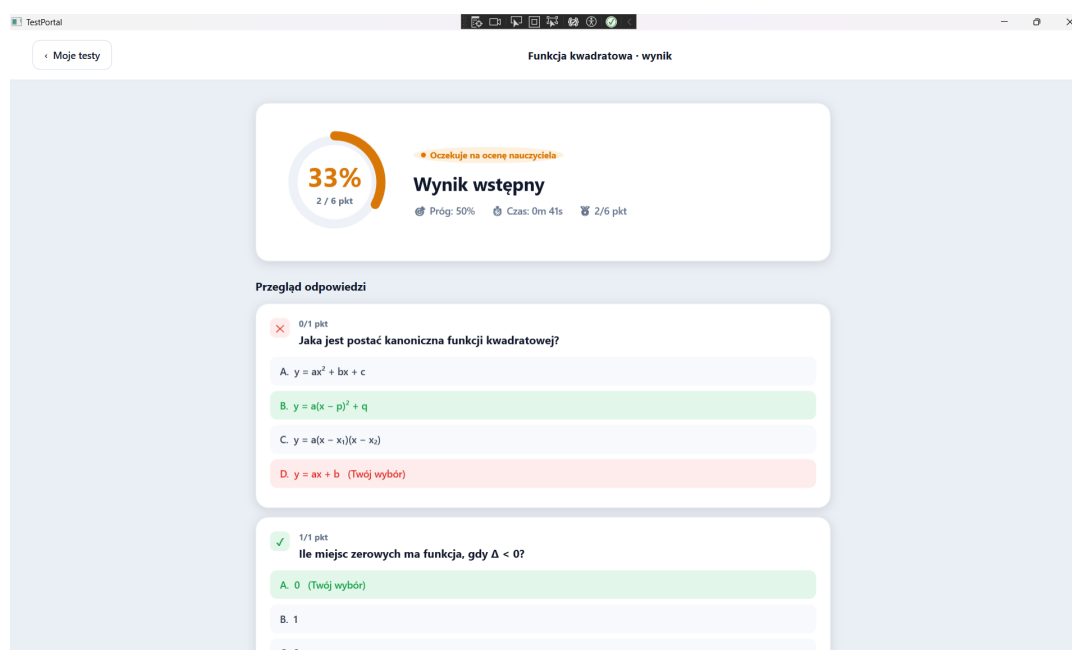
Ekran rozwiązywania testu (rysunek 5.9) prezentuje pojedyncze pytanie wraz z wariantami odpowiedzi lub polem na odpowiedź otwartą. W górnej części znajduje się licznik czasu, a u dołu - nawigacja umożliwiająca przechodzenie między pytaniami oraz pasek postępu. Po upływie czasu test jest automatycznie kończony. Przy oddawaniu testu z pominiętymi pytaniami wyświetlane jest ostrzeżenie. Opuszczenie okna testu jest traktowane jako próba oszustwa.



Rysunek 5.9. Ekran rozwiązywania testu.

5.9. Wynik testu

Po zakończeniu testu uczeń widzi ekran wyniku (rysunek 5.10), prezentujący osiągnięty procent, liczbę zdobytych punktów oraz status (zaliczony, niezaliczony lub oczekujący na ocenę pytań otwartych). Poniżej znajduje się szczegółowy przegląd odpowiedzi - dla każdego pytania wskazane są odpowiedzi poprawne oraz wybrane przez ucznia.



Rysunek 5.10. Ekran wyniku testu wraz z przeglądem odpowiedzi.

6. Podsumowanie

Celem projektu było stworzenie aplikacji wspierającej nauczyciela i ucznia w pełnym cyklu pracy z testami - od ich ułożenia, przez udostępnienie i rozwiązanie, aż po ocenę i analizę wyników. Cel ten został zrealizowany: powstała spójna, w pełni funkcjonalna aplikacja desktopowa napisana w języku C# na platformie .NET, z interfejsem graficznym opartym na technologii WPF i z relacyjną bazą danych PostgreSQL obsługiwaną przez Entity Framework Core.

Zrealizowane funkcjonalności

W ramach projektu udało się zrealizować wszystkie założone funkcje:

- **Konta i role** - rejestracja (z wyborem roli) i logowanie użytkowników z rozróżnieniem ról nauczyciela i ucznia oraz odrębnymi panelami dla każdej z nich; każdy nauczyciel zarządza wyłącznie własnymi testami.
- **Grupy** - tworzenie grup i zarządzanie ich składem, z możliwością przynależności ucznia do wielu grup.
- **Testy** - tworzenie, edycja, duplikowanie i usuwanie testów złożonych z pytań jednokrotnego wyboru, wielokrotnego wyboru oraz pytań otwartych, wraz z walidacją kompletności testu oraz sterowaniem widocznością (test aktywny/nieaktywny).
- **Rozwiązywanie testów** - rozwiązywanie testów w ograniczonym czasie, z obsługą wielu podejść i wyborem najlepszego wyniku.
- **Ocenianie** - automatyczne ocenianie pytań zamkniętych (z punktami częściowymi) oraz ręczne ocenianie i edycja oceny dowolnego podejścia (z możliwością nadpisania punktów każdego pytania) z automatycznym przeliczaniem wyniku.
- **Bezpieczeństwo i integralność** - haszowanie haseł, unikalność loginów, wykrywanie prób oszustwa oraz spójność danych zapewniona przez klucze obce i migracje.
- **Analiza wyników** - statystyki testu z prezentacją wszystkich podejść uczniów oraz eksport wyników do pliku CSV.

W trakcie tworzenia projektu zastosowano zasady programowania obiektowego: dziedziczenie (odwołane w bazie danych strategią Table-per-Hierarchy), enkapsulację oraz podział kodu na niezależne, logiczne warstwy. Dzięki temu struktura aplikacji jest czytelna i łatwa w utrzymaniu.

Możliwości dalszego rozwoju

Choć aplikacja jest w pełni funkcjonalna, można ją w przyszłości rozbudować, na przykład o:

- losowanie kolejności pytań i odpowiedzi w celu dodatkowego utrudnienia ściągania,
- podgląd historii własnych podejść po stronie ucznia (obecnie uczeń widzi swój najlepszy wynik, a pełną historię podejść ma nauczyciel w statystykach),

- terminy dostępności testów (przedział dat, w którym test można rozwiązać),
- rozbudowane raporty i wykresy postępów uczniów w czasie.

Dodanie tych funkcji zwiększyłyby użyteczność systemu, jednak w obecnej formie projekt w pełni spełnia założone wymagania i stanowi solidną podstawę do dalszej rozbudowy.

Bibliografia

- [1] Microsoft. Dokumentacja języka c#. <https://learn.microsoft.com/dotnet/csharp/>, 2026. dostęp: 2026.
- [2] Microsoft. Entity framework core – dokumentacja. <https://learn.microsoft.com/ef/core/>, 2026. dostęp: 2026.
- [3] Microsoft. Windows presentation foundation (wpf) – dokumentacja. <https://learn.microsoft.com/dotnet/desktop/wpf/>, 2026. dostęp: 2026.
- [4] Npgsql Development Team. Npgsql – .net data provider for postgresql. <https://www.npgsql.org/>, 2026. dostęp: 2026.
- [5] The PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026. dostęp: 2026.

Spis rysunków

3.1	Diagram ERD bazy danych aplikacji.	11
3.2	Diagram UML klas modelu aplikacji.	13
4.1	Diagram Gantta przedstawiający harmonogram realizacji projektu.	15
5.1	Ekran logowania i rejestracji.	17
5.2	Pulpit nauczyciela.	18
5.3	Lista testów nauczyciela ze statusem widoczności i akcjami.	19
5.4	Edytor testu wraz z ustawieniami i listą pytań.	19
5.5	Ocenianie pytań otwartych.	20
5.6	Widok zarządzania grupami uczniów.	21
5.7	Statystyki wyników testu.	22
5.8	Lista testów przypisanych uczniowi.	22
5.9	Ekran rozwiązywania testu.	23
5.10	Ekran wyniku testu wraz z przeglądem odpowiedzi.	24

Spis tabel

3.1	Najważniejsze tabele bazy danych aplikacji.	11
-----	---	----